

Docker Implementation Guide

AgriTech Marketplace - VM deployment, Docker Compose, Nginx frontend, Spring Boot backend, and MySQL

Project: AgriTech Marketplace Docker Deployment

Scope: Implementation guide, verification, and troubleshooting notes.

1. What this guide covers

This implementation guide documents the Docker-based deployment of the AgriTech Marketplace on an Ubuntu VM. It covers the frontend container, Spring Boot backend container, MySQL container, Docker Hub publishing, and the final validation steps.

2. Final architecture

- Frontend container on port 80 serves the static website.
- Backend container on port 8080 runs the Spring Boot application.
- MySQL container on port 3306 stores application data.
- Docker Compose connects the three containers on one bridge network.
- Docker Hub stores the built images for reuse and sharing.

3. Build flow

The deployment followed this flow:

1. Pull the latest code from GitHub into /opt/agritech.
2. Build the backend JAR with Maven inside the Dockerfile.
3. Build the frontend Nginx image with the updated nginx.conf.
4. Start all services with docker compose up -d --build.
5. Verify containers, ports, logs, and browser access.

4. Dockerfiles and compose layout

Backend Dockerfile pattern:

```
FROM maven:3.9.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]
```

Frontend Dockerfile pattern:

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

EXPOSE 80

docker-compose.yml pattern:

```
services:
  mysql:
    image: mysql:8.0
    container_name: agritech-mysql
    ports:
      - "3306:3306"
  backend:
    build: ./backend
    container_name: agritech-backend
    ports:
      - "8080:8080"
    depends_on:
      - mysql
  frontend:
    build: ./frontend
    container_name: agritech-frontend
    ports:
      - "80:80"
    depends_on:
      - backend
```

5. Key configuration decisions

- The frontend was configured as an Nginx container, which is ideal for static HTML, CSS, and JavaScript.
- The backend was packaged as a Java JAR and exposed on port 8080.
- Nginx was updated so `/api/*` requests can be forwarded to the backend when needed.
- MySQL was kept as a separate container so the application state stays isolated from the app code.

6. Verification commands

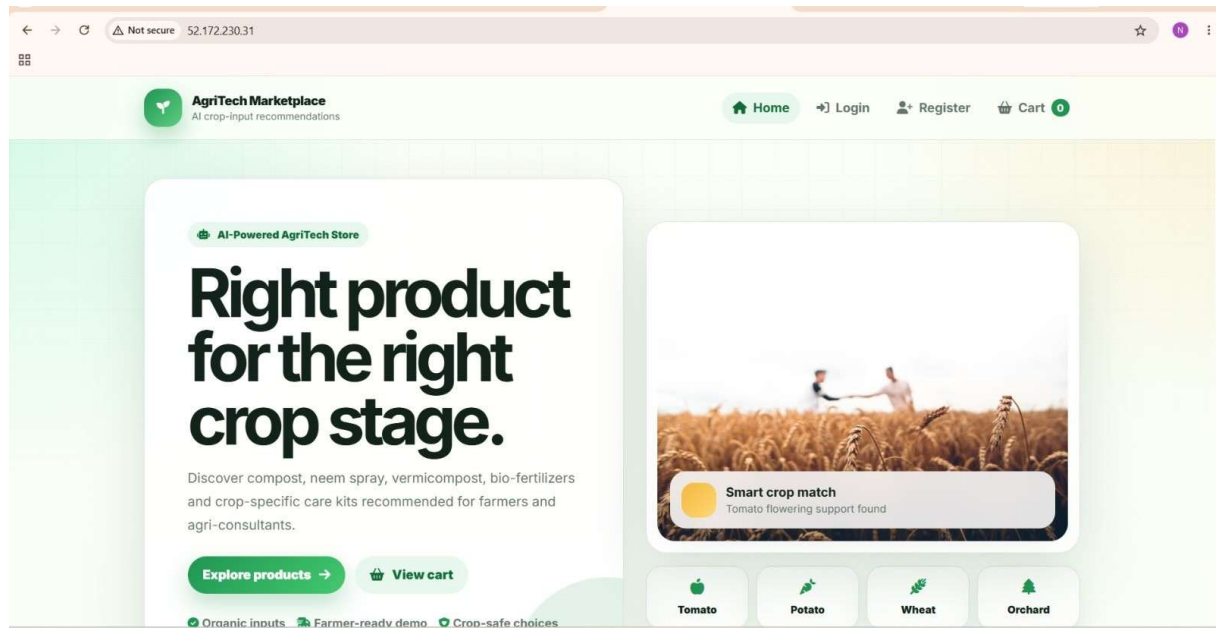
```
docker ps
ss -tulpn | grep :80
ss -tulpn | grep :8080
ss -tulpn | grep :3306
docker logs agritech-backend --tail 50
docker logs agritech-frontend --tail 50
```

These checks confirmed that Docker published the ports correctly and that each container stayed up after rebuilds.

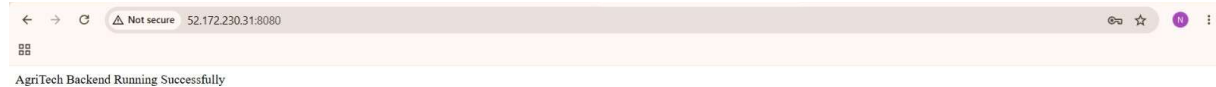
7. Evidence from the VM

```
root@agritech-dev-vm:/opt/agritech/docker-compose# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        NAME
STATUS        PORTS
4308757a49c0   docker-compose_frontend             "/docker-entrypoint.s... 54 seconds ago   agri
Up 52 seconds  0.0.0.0:80->80/tcp, [::]:80->80/tcp
tech-frontend
6d9fdefc9f9c   docker-compose_backend             "java -jar app.jar"      55 seconds ago   agri
Up 53 seconds  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
tech-backend
6790a7279747   mysql:8.0                          "docker-entrypoint.s... 56 seconds ago   agri
Up 54 seconds  0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp
tech-mysql
root@agritech-dev-vm:/opt/agritech/docker-compose#
```

Container status after docker compose up -d --build



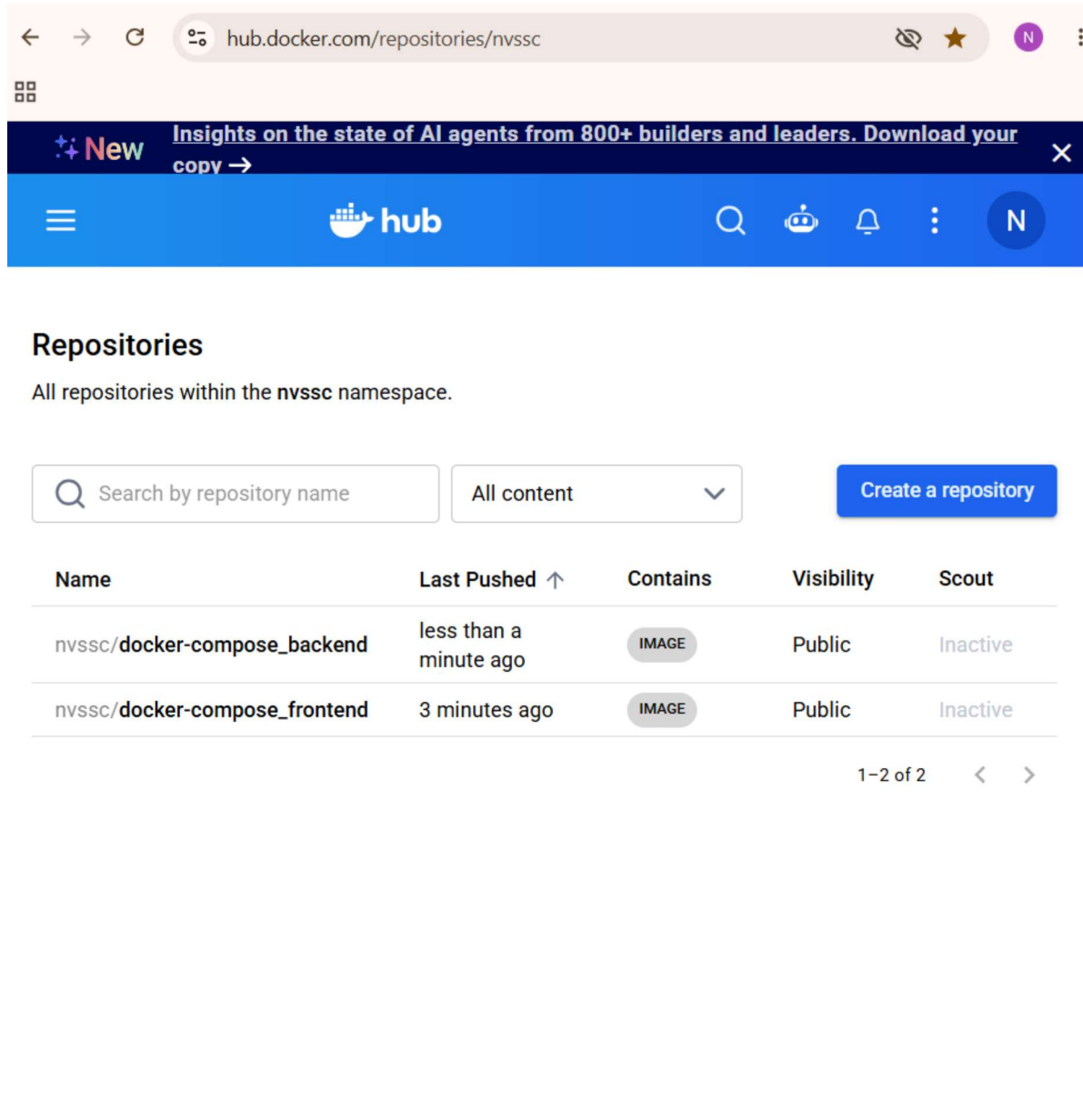
Frontend application served on port 80



Backend health response on port 8080

8. Docker Hub publishing

The built images were pushed to Docker Hub so they can be shared and reused. The repository namespace used in the screenshots was nvssc.

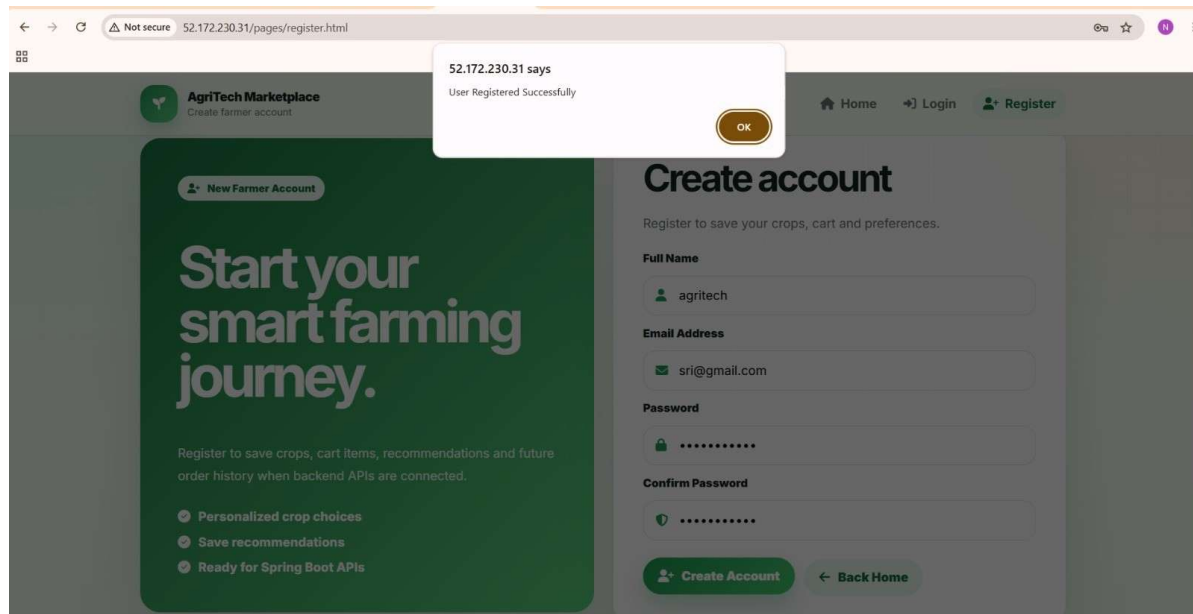


Docker Hub repositories created for backend and frontend images

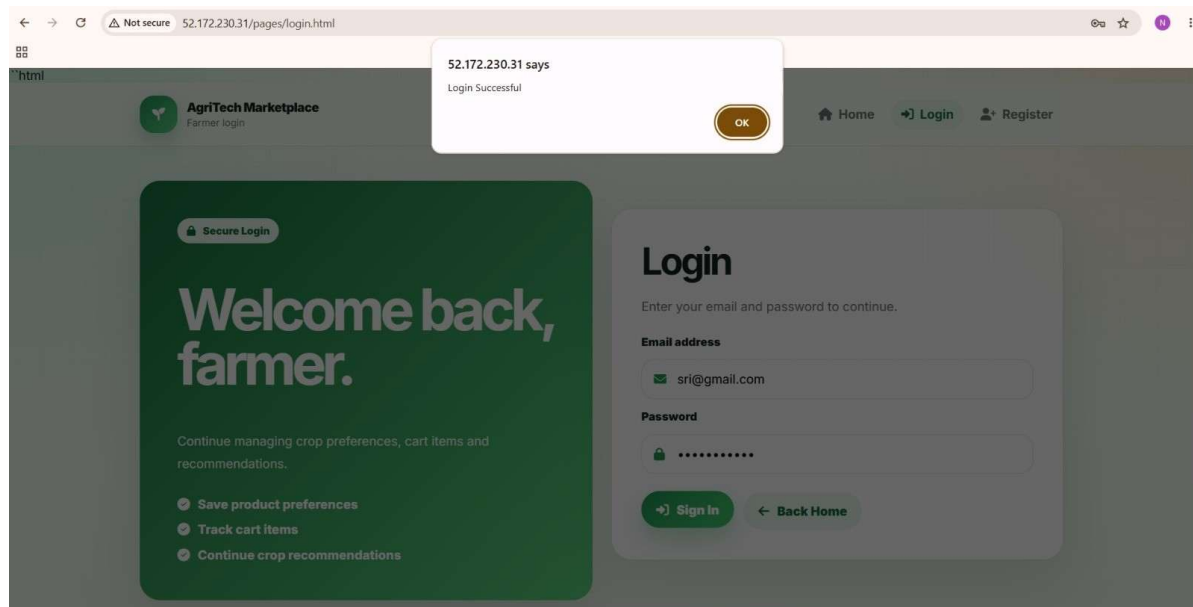
9. Deployment result

- The browser opened the frontend successfully on the public VM IP.
- Registration and login flows returned success messages.

- The backend responded on port 8080, proving the container and application startup were correct.
- Docker Hub also showed the pushed image repositories.



Register flow working in the browser



Login flow working in the browser

10. Commands used during the final validation

```
cd /opt/agritech/docker-compose
docker-compose down
git reset --hard HEAD
git pull
```

```
docker-compose up -d --build  
docker ps  
cloud-init status
```

11. Outcome

The final setup is a working Docker-based three-container deployment for the AgriTech Marketplace, with the frontend on port 80, backend on port 8080, and MySQL on port 3306. The same container images were also published to Docker Hub for sharing.